Problemas de Otimização

Vítor Gauer Pereira

Programa de Pós Graduação em Computação Aplicada - PPGCA

UDESC
UNIVERSIDADE DO ESTADO DE SANTA CATARINA

Sumário

1. Problemas de Otimização
2. Solução de Problemas de Otimização
3. Heurística Construtiva
4. Heurística de Refinamento
5. Meta-heurísticas
6. Algoritmo Genético

Problemas de Otimização

Problema de Otimização

*Em ciência da computação, um **problema** é uma tarefa que um algoritmo pode resolver ou responder*

Problemas solucionados computacionalmente:

1. Ordenação
2. Busca
3. Contagem
4. Decisão
5. **Otimização**

Problemas de Otimização é uma classe de problemas resolvida por algum método de otimização que busca encontrar uma solução ótima em um vasto espaço de possibilidades de soluções. Assim, a saída para um problema de otimização é a solução em si.

Áreas de Aplicação



Problemas:

- Caixeiro Viajante
- Roteamento de Veículos
- Problema da Mochila
- Problema de Alocação de Recursos/Pessoas
- Problema de Empacotamento *(Bin Packing)*
- Portfólio de Investimentos

…

Áreas:

- Logística
- Transporte
- Agronegócio
- Aviação
- Petróleo
- Finanças

…







Caixeiro Viajante

- Cidades a serem visitadas
- Encontrar a melhor rota que visite cada cidade uma única vez e retorne para a cidade de origem





Problema de Otimização

$$P = (S, f, \Omega)$$

- Função objetivo *(f)*
  - A qual deve ser **maximizada** ou **minimizada** (Ex: max de lucro ou min de custo)
  - Pode não existir ou ser mais de uma
  - Forma de associar um valor a uma possível solução (qualidade da solução)

- Conjunto de variáveis de decisão (S)
  - Variáveis que **afetam o valor da função objetivo** {x1, x2, x3, …, xn}
  - Em problemas complexos esse conjunto pode ser muito grande

- Conjunto de restrições (Ω)
  - Não permite que a o conjunto de variáveis assuma determinados valores
  - Uma solução não satisfazer as restrições do problema, ela é considerada não factível

Representação Matemática

$$\underset{\boldsymbol{x}\in\Re^n}{\text{minimize}} \quad f_i(\boldsymbol{x}), \qquad (i = 1, 2, ..., M),$$

$$\text{subject to} \quad h_j(\boldsymbol{x}) = 0, \quad (j = 1, 2, ..., J),$$

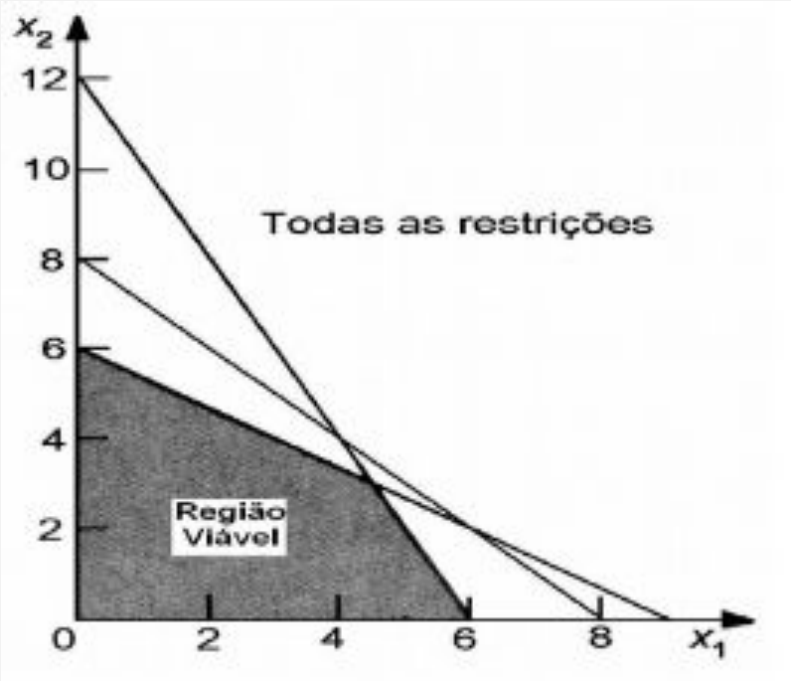
$$g_k(\boldsymbol{x}) \leq 0, \quad (k = 1, 2, ..., K),$$

where $f_i(\boldsymbol{x}), h_j(\boldsymbol{x})$ and $g_k(\boldsymbol{x})$ are functions of the design vector

$$\boldsymbol{x} = (x_1, x_2, ..., x_n)^T.$$

Região Factível

- Região do espaço de busca que se encontram todas as soluções viáveis
- Onde não existe infração das restrições
- Região comum a todas as restrições



Exemplo de Solução Factível

Problema da Padaria (Croissants e Strudels): tem um lucro de 20 centavos por *croissant* e 50 centavos por *strudel.* Sejam c e s o número de Croissants e Strudels, respectivamente.

Lucro: 0.2c + 0.5s

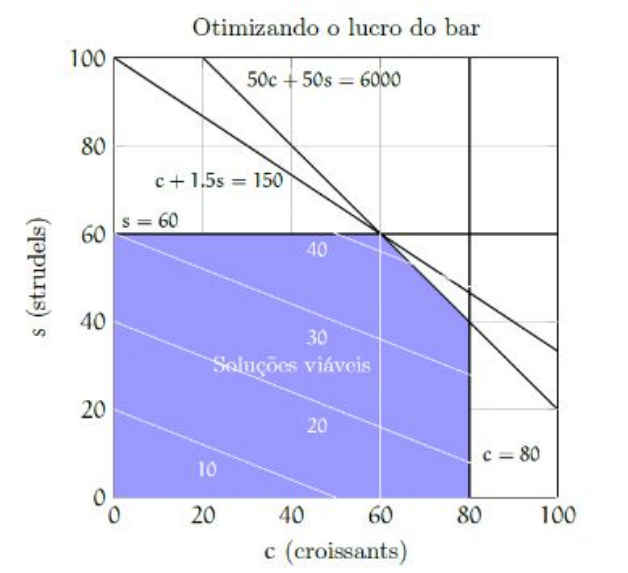
- Diariamente até 80 clientes compram um *Croissant* e até 60 um *Strudel*
- Dispõe apenas de 150 ovos e 6kg de açúcar por dia
- Para o Croissant preciso de 1 ovo e 50g de açúcar
- Para o Strudel preciso de um 1 e meio e 50g de açúcar

$$
\begin{aligned}
\textbf{maximiza} \quad & 0.2c + 0.5s, \\
\textbf{sujeito a} \quad & c + 1.5s \leq 150, \\
& 50c + 50s \leq 6000, \\
& c \leq 80, \\
& s \leq 60, \\
& c, s \geq 0.
\end{aligned}
$$

Quantos Croissants e Strudels a padaria deve produzir para obter o MAIOR LUCRO?

Exemplo de Solução Factível

$$
\begin{aligned}
\textbf{maximiza} \quad & 0.2c + 0.5s, \\
\textbf{sujeito a} \quad & c + 1.5s \leq 150, \\
& 50c + 50s \leq 6000, \\
& c \leq 80, \\
& s \leq 60, \\
& c, s \geq 0.
\end{aligned}
$$

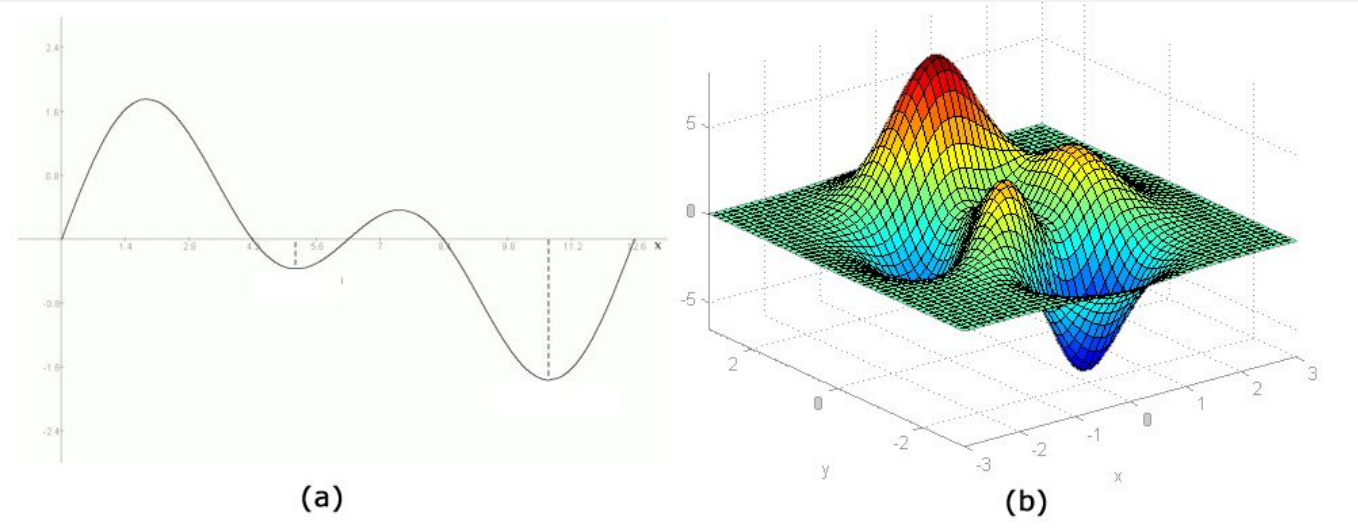


Processo de Busca

- Solucionar um problema de otimização requer **identificar uma solução** particular em um **espaço de possibilidades**
- Esse espaço é enorme para **problemas do mundo real**
- O processo de solução pode ser visto como uma **busca** em um conjunto muito grande de **possibilidades de solução** para encontrar a melhor delas

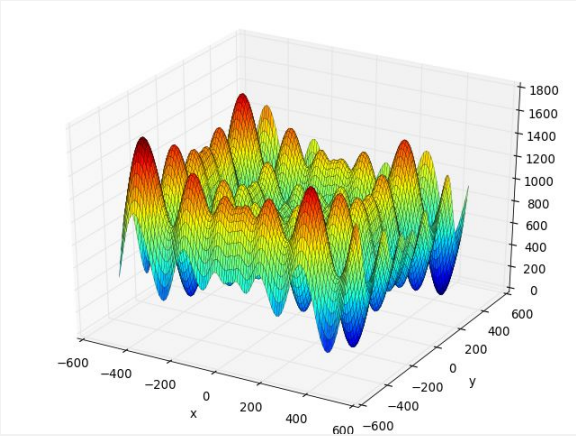
Espaço de Busca

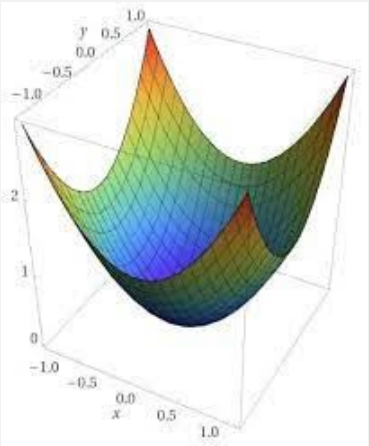
Espaço no qual o algoritmo de otimização deverá realizar a busca pela solução ótima ou pelas soluções ótimas



Problema Multimodal

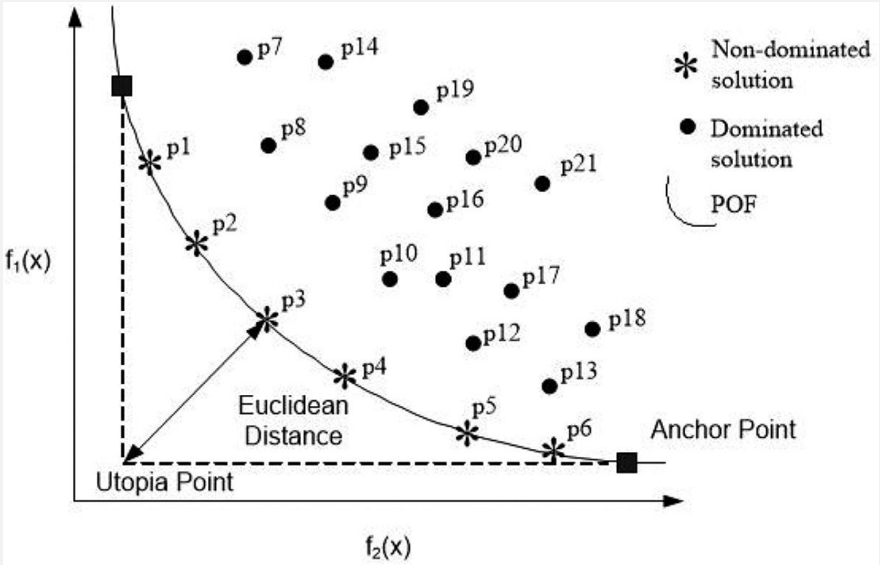
- **Multimodal:** Existe um número de pontos que são melhores que todas as suas soluções vizinhas
- Esse pontos são **Ótimos Locais** e o mais alto (maximização) ou mais baixo (minimização) é chamado de **Ótimo Global**
- Um problema com apenas um ótimo local é chamado de **unimodal**





Problemas Multiobjetivo

- Problemas que envolvem **mais de um objetivo** a ser maximizado/minimizado
- Resultado é um **conjunto de soluções** (fronteira/solução de Pareto)
- Não existe apenas uma solução ótima
- Exemplo: Portfólio de Investimentos
  - max lucros & min riscos



Domínios do Problema

- Otimização Contínua
  - Variáveis assumem valores reais (ou contínuos)

- **Otimização Combinatória ou Discreta**
  - Variáveis assumem valores discretos (ou inteiros)

- Otimização Mista
  - Variáveis inteiras e contínuas

Função Objetivo e Satisfação de Restrições

- **Problemas de Otimização com Restrições**
  - *COP - Constrained Optimization Problem*
  - Com função objetivo e restrição

- Problemas de Satisfação de Restrições
  - *CSP - Constraint Satisfaction Problem*
  - Não possui função objetivo, apenas restrições

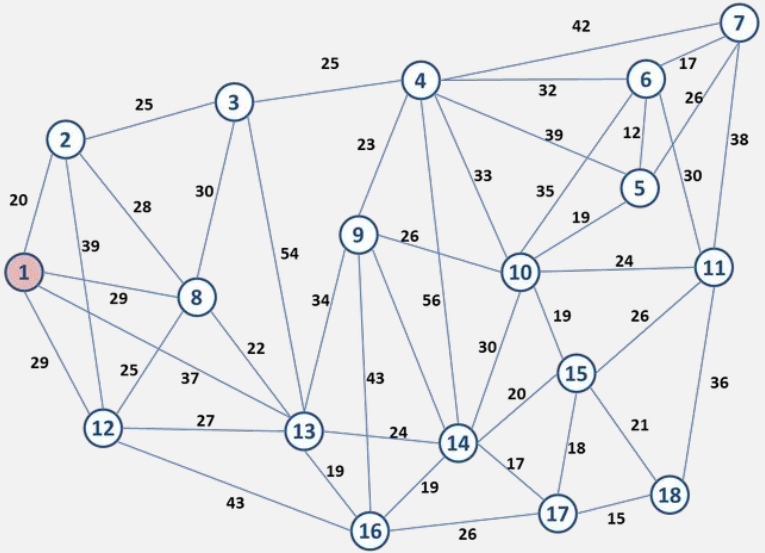
- Problemas Livres de Restrições
  - *FOP - Free Optimization Problem*
  - Não possui restrição, apenas função objetivo

	Objective function	
Constraints	Yes	No
Yes	Constrained optimisation problem	Constraint satisfaction problem
No	Free optimisation problem	No problem

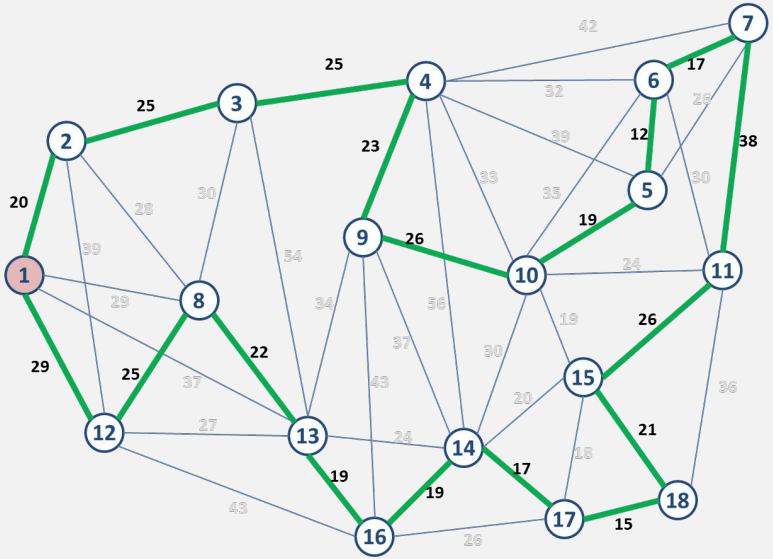
Problema de Otimização: dentre as soluções viáveis, qual a melhor?

Caixeiro Viajante

Entrada



Solução



OBS: pode ser aplicado para outros problemas, com sequenciamento de tarefas…

Caixeiro Viajante



Caixeiro Viajante - Modelagem

Considere um grafo *G = (Cidades, A)*, em que Cidades são os locais (clientes) a serem visitados e *A* o conjunto de arestas que ligam duas cidades, isto é, *A = {(i,j) | i ≠ j}*. Seja *cij* o custo (distância, tempo, etc) de ir da cidade *i* para a cidade *j*.

- Variáveis de decisão: xij é a variável binária que indica se a aresta *(i,j)* foi utilizada.
  - xij = 1, indica que a cidade *j* é atendida logo após a cidade *i*
  - xij = 0, caso contrário

$$\text{minimizar } \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij}x_{ij}$$

FO: minimizar soma dos custos

sujeito a :

A cada cidade só chega uma aresta

$$\sum_{i=1}^{n} x_{ij} = 1, \qquad \forall j = 1,\ldots,n$$

A cada cidade só sai uma aresta

$$\sum_{j=1}^{n} x_{ij} = 1, \qquad \forall i = 1,\ldots,n$$

Evita Subciclos

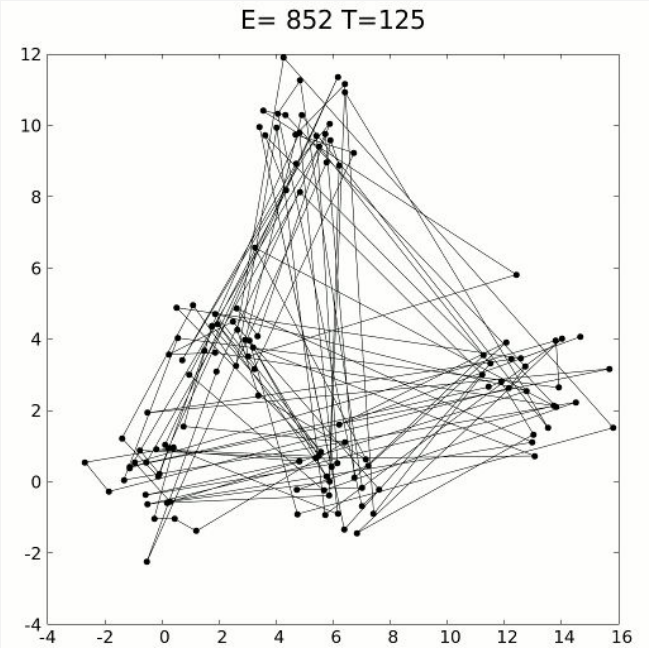
$$\sum_{i\in S}\sum_{j\in N-S} x_{ij} \geq 1, \quad \forall S \subset N, S \neq \{\}, S \neq N$$

Domínio das variáveis

$$x_{ij} \in \{0,1\}, \qquad \forall i = 1,\ldots,n;\ j = 1,\ldots,n$$

Restrições

Caixeiro Viajante



Caracterização de um Problema - Abstração



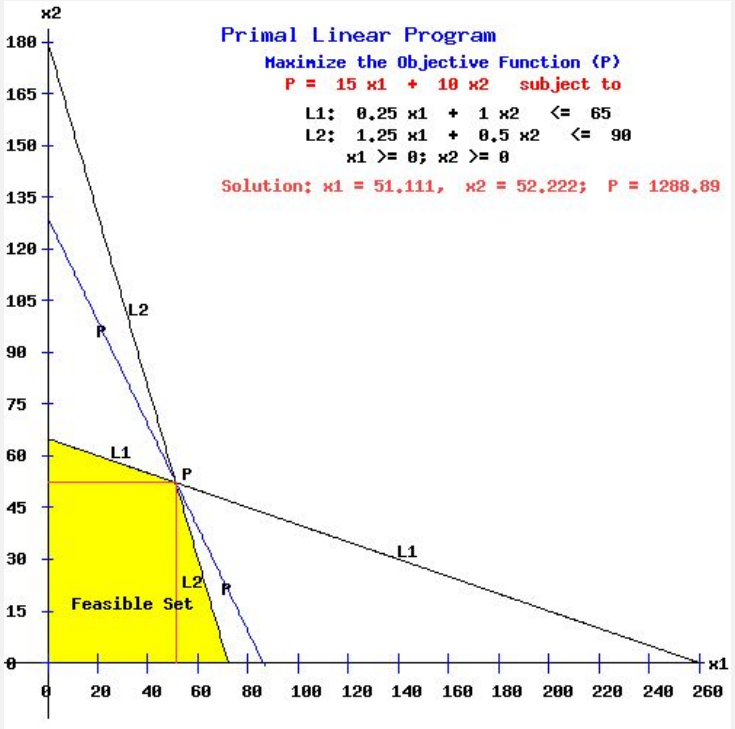
Solução de Problema de Otimização

Métodos de Otimização

- **Determinísticos:** alcançam a solução por meio de aplicação direta de uma série de etapas pré-definidas
  - Sempre a mesma solução para a mesma entrada
  - Método analítico
  - Métodos enumerativos (exaustivo) - Métodos exatos
  - Heurísticas (Construtiva ou Refinamento)
  - Outros…
- **Métodos Estocásticos / Probabilísticos (Meta-heurísticas):** introduzem algum nível de aleatoriedade durante a busca por uma solução
  - Solução diferente para a mesma entrada
  - Busca aleatória
  - Computação Evolucionária
  - Inteligência de Enxames
  - Outros…

Determinístico - Método Analítico

- **Em problemas de otimização muito simples,** em que podem ser definidos por equações lineares ou quadráticas
- A melhor solução pode ser encontrada pela **análise direta ou resolução das funções**
- Impraticável para problemas muito complexos e com entradas grandes



Determinístico - Método Exato

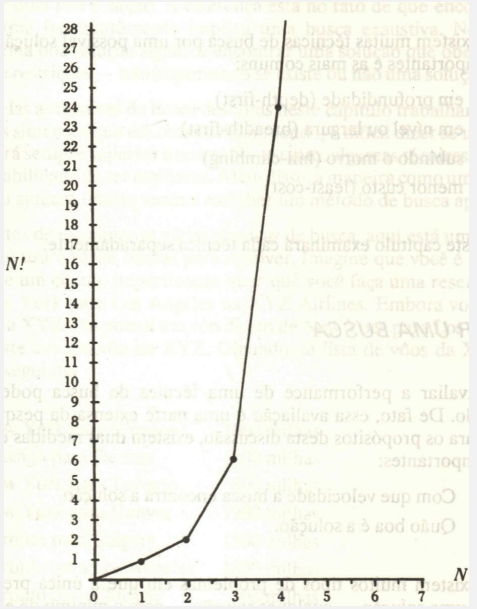
- Abordagem que tem objetivo de **avaliar todas as possíveis soluções** para encontrar a melhor delas
- Baseado na modelagem matemática
- Inviável para cenários muito complexos com entradas muito grandes devido a **explosão combinatória de possibilidades**
- Em cenários reais, levariam anos para encontrar a solução ótima
- Assim, estaria **além dos limites** de processamento até do computador mais poderoso que existe atualmente

Determinístico - Método Exato

Caixeiro Viajante - Explosão combinatória

Para N cidades tenho (N-1)! possíveis combinações de rotas

- 11 cidades 10! = 3.628.800 rotas
- 12 cidades 11! = 39.916.800 rotas
- 21 cidades 20! = 2 x 10^18 rotas
- 101 cidades 100! = 9 x 10^157 rotas



Inviável resolver por meio de métodos exatos

Determinístico - Heurística

- Devido às limitações dos métodos analíticos e exatos para casos considerados grandes, surgiram as heurísticas para **viabilizar solução** para estes cenários

- Aplicam regras gerais e simples, sem a aplicação de método exaustivo
  - Utiliza conhecimento específico do problema para solucioná-lo

- São métodos **aproximativos** que buscam obter solução de **boa qualidade em tempo viável**

- Não tem a obrigação de atingir a melhor solução possível e não necessita de prova matemática

- Heurísticas Construtivas e Heurísticas de Refinamento

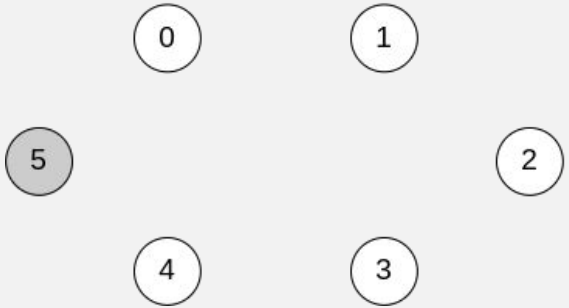
Heurística Construtiva - Vizinho Mais Próximo

- Heurística construtiva **gulosa**: observa o passo a seguir **mais econômico**
- **Específica para problemas de grafo** como Caixeiro Viajante
- Construir uma rota passo a passo, adicionando a cidade mais próxima (ainda não visitada) da última cidade inserida
- O procedimento encerra quando todas as cidades forem visitadas e retornar para a origem

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0



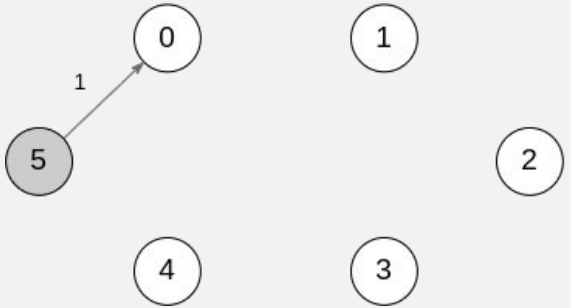
Distância = 0

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

i	j	d_{ij}
5	0	1
5	1	2
5	2	6
5	3	6
5	4	2



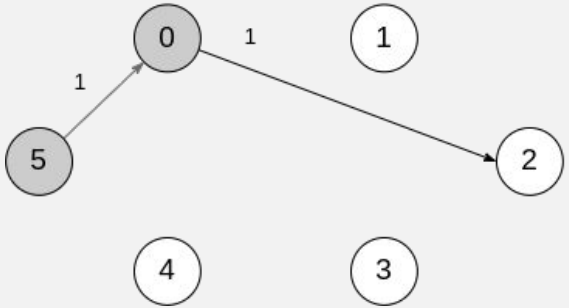
Distância = 1

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

i	j	d_{ij}
0	1	2
0	2	1
0	3	4
0	4	9



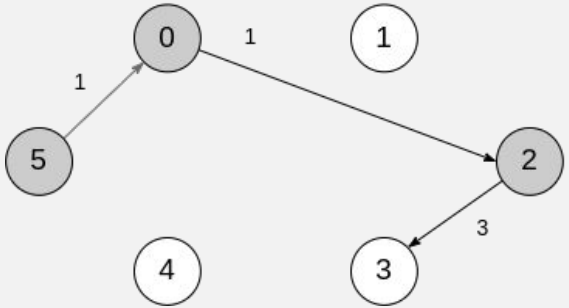
Distância = 1+1 = 2

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

i	j	d_{ij}
2	1	5
2	3	3
2	4	8



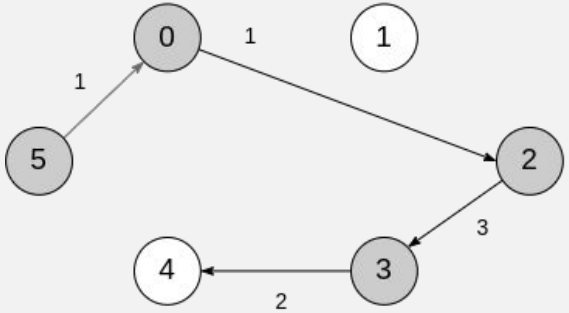
Distância = 2+3 = 5

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

i	j	d_{ij}
3	1	9
3	4	2



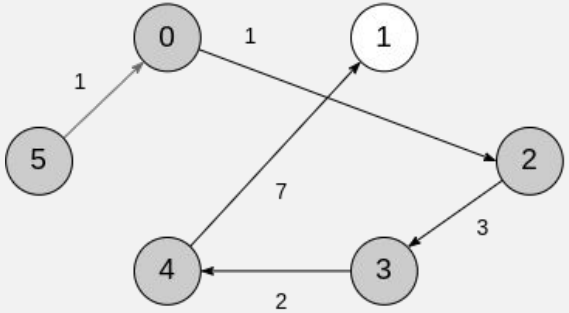
Distância = 5+2 = 7

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

i	j	d_{ij}
4	1	7



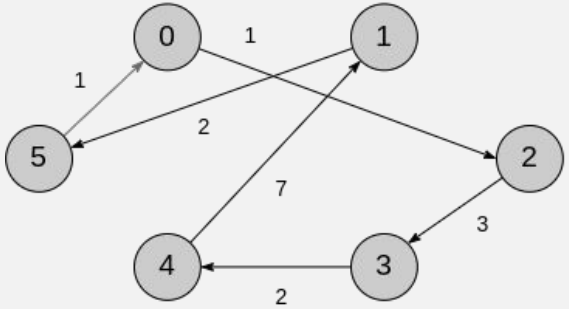
Distância = 7+7 = 14

Heurística Construtiva - Vizinho Mais Próximo

Matriz de distância (Simétrica)

Cid	0	1	2	3	4	5
0	0	2	1	4	9	1
1	2	0	5	9	7	2
2	1	5	0	3	8	6
3	4	9	3	0	2	6
4	9	7	8	2	0	2
5	1	2	6	6	2	0

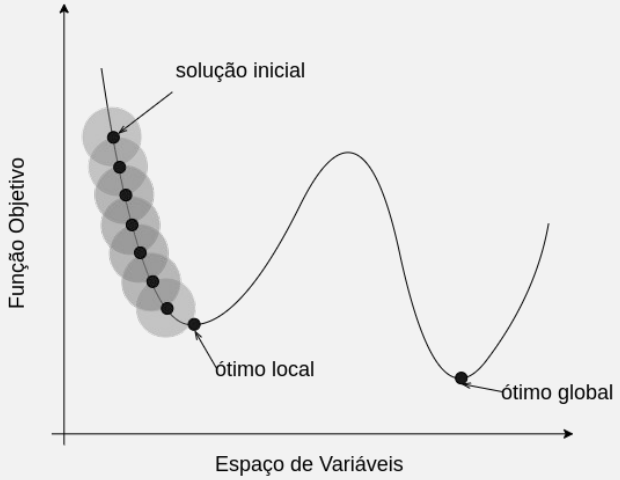
Inserção forçada para a origem



Distância = 14+2 = 16

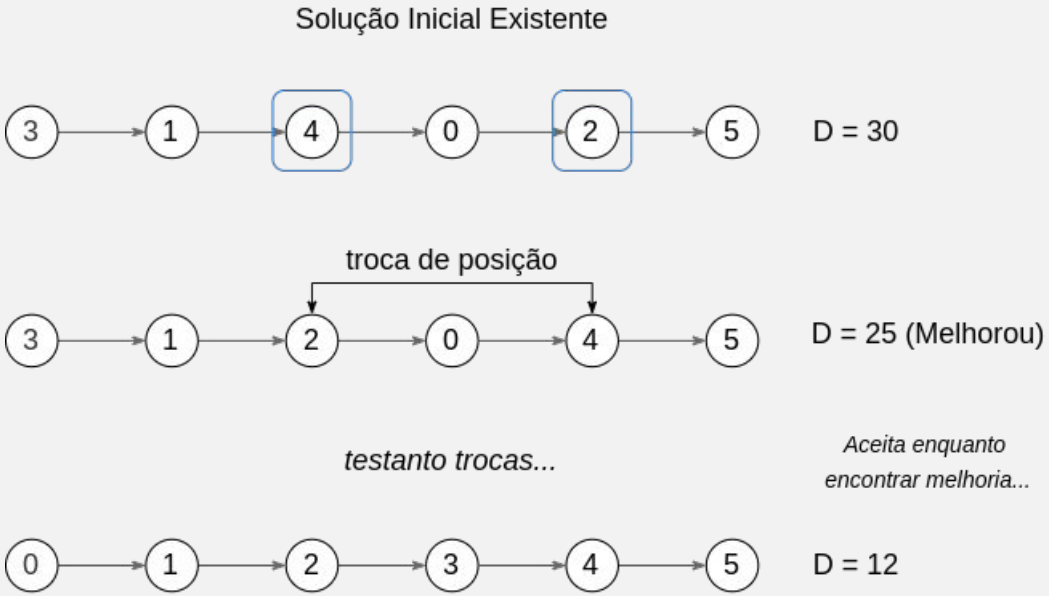
Heurística de Refinamento

- Também chamadas de **heurísticas de busca local**
- Baseada na **noção de vizinhança**
- A cada solução existente ***S*** está associado um conjunto de **vizinhos *S'*** definidos por um determinado **movimento** (algoritmo)
- O movimento depende do problema tratado



Heurística de Refinamento - Exchange

Para o problema do Caixeiro Viajante podemos aplicar o operador de ***Exchange***



Limitação dos Métodos Determinísticos

- Métodos Exatos e Analíticos não podem ser aplicados para **cenários reais**

- Heurísticas são métodos muito simples

    - **Não prevê os impactos** do passo recém executado
    - Escolhe o caminho mais econômico a primeira vista
    - Não necessariamente conduz a busca a uma direção promissora
    - **Não faz exploração** do espaço de busca
    - **Não são robustas** o suficiente

Estocásticos - Busca Aleatória

- Testar **soluções aleatórias** e considerar a melhor solução encontrada
  - Ex: Caixeiro Viajante: gerar n rotas aleatoriamente

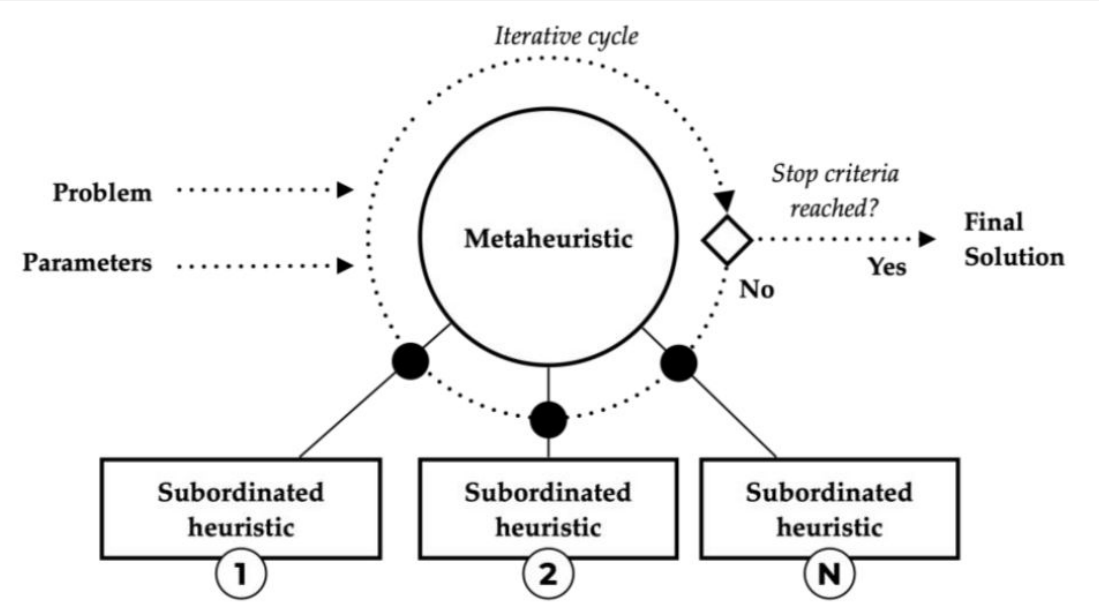
- Útil quando se sabe pouco sobre a relação entre os dados de entrada e a função objetivo

- Abordagens puramente aleatórias são normalmente chamadas de métodos de Monte Carlo (inspirado na confiança na sorte exibida nos cassinos)

Estocásticos - Meta-heurísticas

- **Limitações do uso apenas de heurísticas:** muito simples, específica e não tão robusta
- **Meta-heurísticas: Combinações de métodos heurísticos** básicos em arquiteturas de alto nível com objetivo de promover melhor exploração do espaço de busca
- Conjunto de regras que orientam o processo de otimização
- Neste caso, as regras são ‘meta-’ porque são aplicadas a um algoritmo geral em vez de diretamente a um problema particular
- **Generalizáveis** para qualquer problema de otimização

Estocásticos - Meta-heurísticas



Estocásticos - Meta-heurísticas

- **Utiliza operações aleatórias ou probabilísticas** e por isso não garante chegar na mesma solução a cada execução

- **Também não há garantia de solução ótima**, porém é o método mais eficaz atualmente para encontrar soluções em tempo razoável

- Inspiradas em **processos encontrados na natureza**, pois a natureza em boa em resolver problemas
  - Processo Evolutivo (Seleção Natural)
  - Formigas / Abelhas ao encontrar melhor caminho na busca por alimento
  - Comportamento de enxames (coordenação e conjunto)







Conceitos Importantes

- **Busca global** (Exploration/Diversificação): promove uma maior amplitude na busca por regiões promissoras.

- **Busca local** (Exploitation/Intensificação): refina a busca em uma região específica do espaço de soluções.

- **Métodos de solução única:** Minimizam uma única solução a cada execução do algoritmo.

- **Métodos baseados em população:** Utilizam um conjunto de soluções (população) para realizar a busca no espaço de soluções.

- Formas de criação da solução:
  - **Algoritmos de construção:** iterativamente o algoritmo “monta”/constrói uma solução para o problema
  - **Algoritmos de melhoria:** o algoritmo parte de uma possível solução do problema e tenta melhorá-la a cada iteração

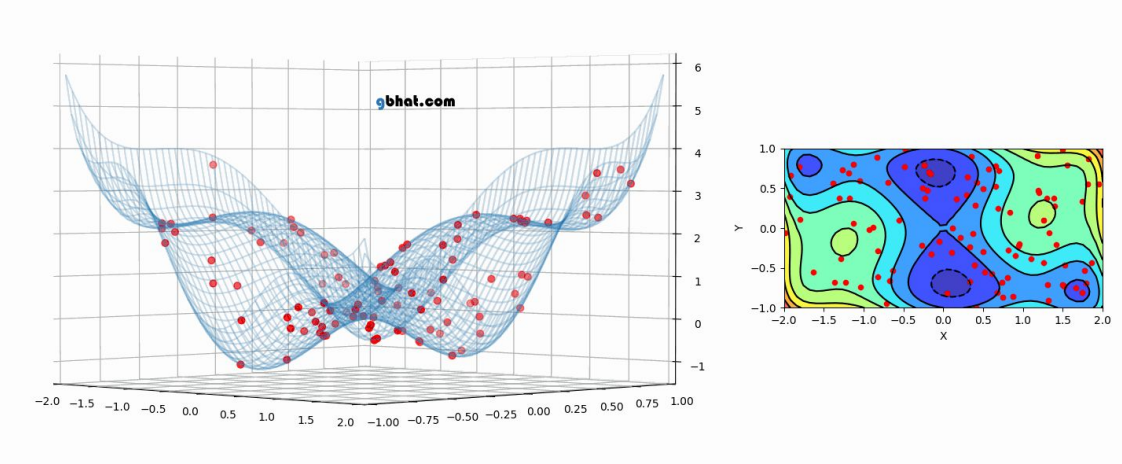
Compartilhamento de Informações

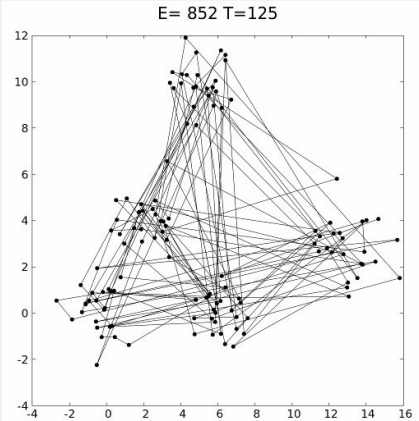
- Meta-heurística possui maneiras de **compartilhamento de informações entre as soluções**, de maneira direta ou indireta
  - **Direta:** combinação entre duas soluções
  - **Indireta:** usa dados acumulados ao longo da busca para a geração de novas soluções

- Com o compartilhamento de informações, soluções melhores são geradas a cada passo, isso se chama **CONVERGÊNCIA**

- Ao final da busca o algoritmo deve ter convergido para uma solução em comum

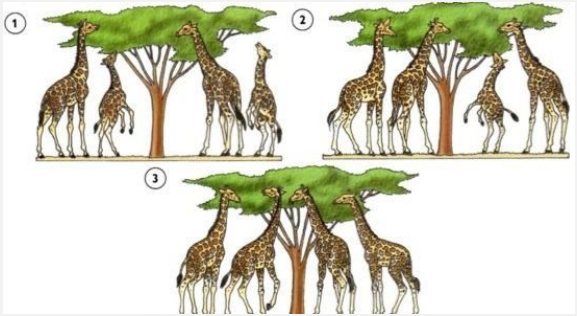
Convergência





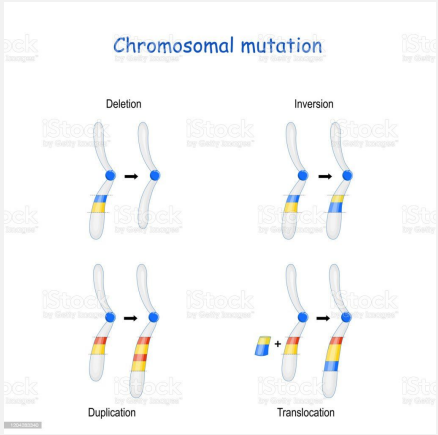
Algoritmo Genético - Inspiração

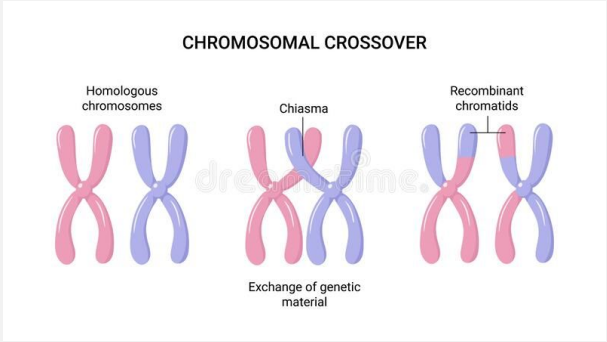
- Baseado na Seleção Natural de Darwin
- **Herança:** filhos recebem propriedades dos seus pais
- **Variação:** variedade de características presentes na população e meios de introduzir variação
- **Seleção:** mecanismo de impedir que determinados indivíduos “sobrevivam” (o mais forte sobrevive)



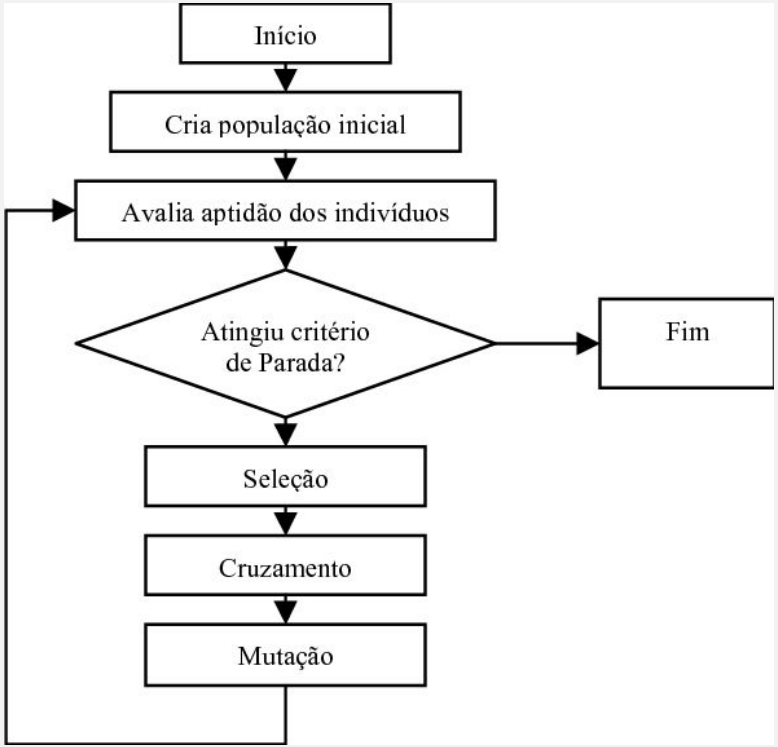
Algoritmo Genético - Terminologias

- **Cromossomo/Indivíduo:** representação de uma solução para o problema
- **População:** conjunto de indivíduos
- **Geração:** maneira de identificar diferentes populações ao longo do processo evolutivo
- **Fitness:** função que avalia a qualidade de um indivíduo perante ao problema tratado
- **Crossover:** reprodução dos indivíduos, gerando novos indivíduos a partir de 2 pais
- **Mutação:** pequenas perturbações aleatórias para gerar variações nos indivíduos
- **Seleção:** escolha dos indivíduos mais bem adaptados para gerar “filhos”

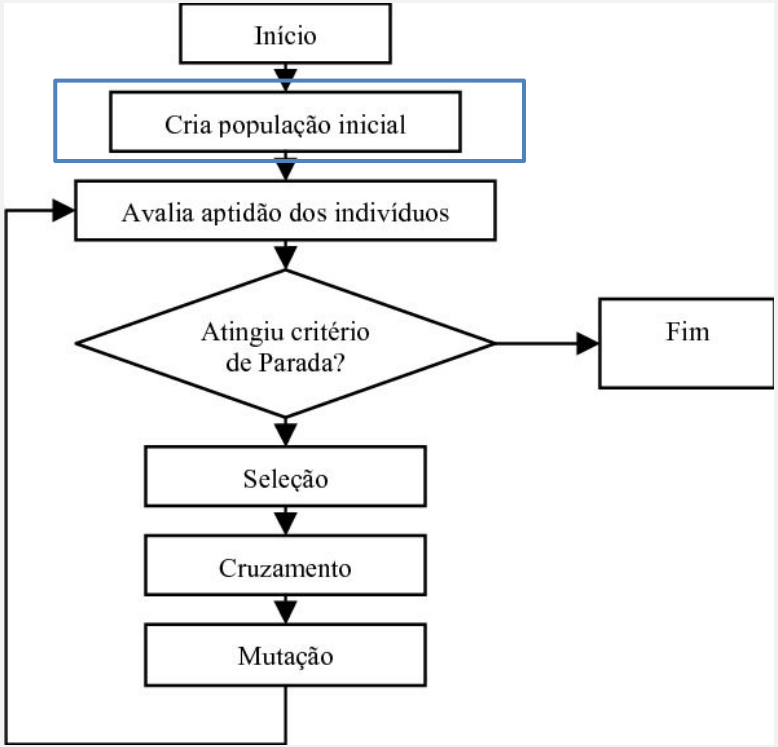




Algoritmo Genético

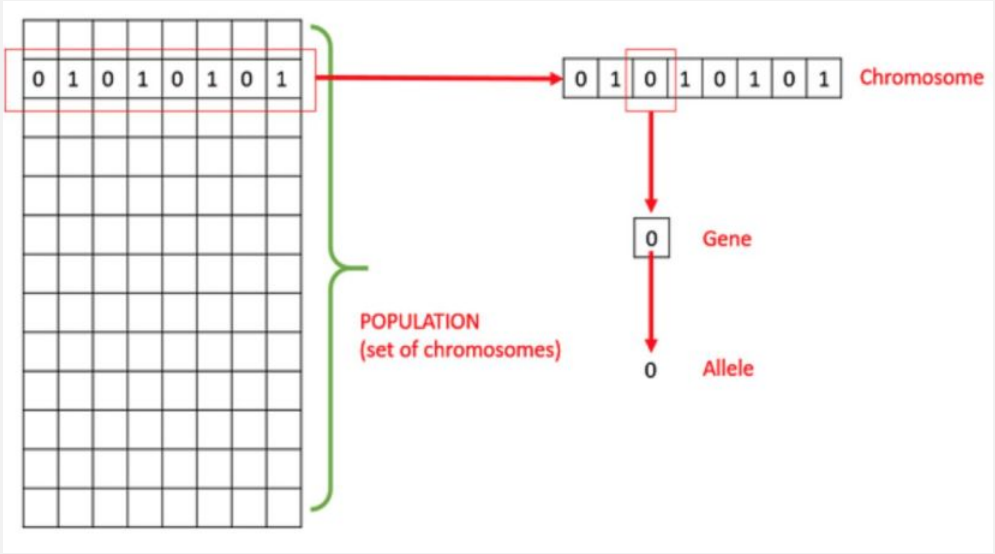


Algoritmo Genético



Algoritmo Genético - População Inicial

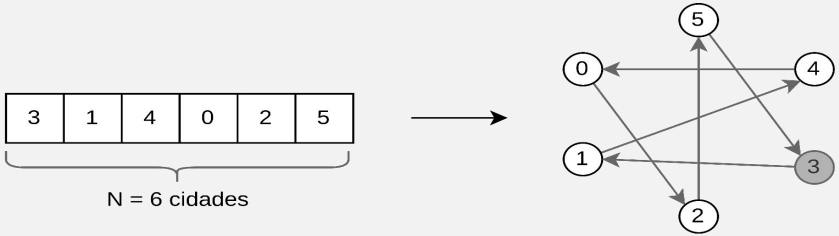
População Inicial de Indivíduos / Cromossomos

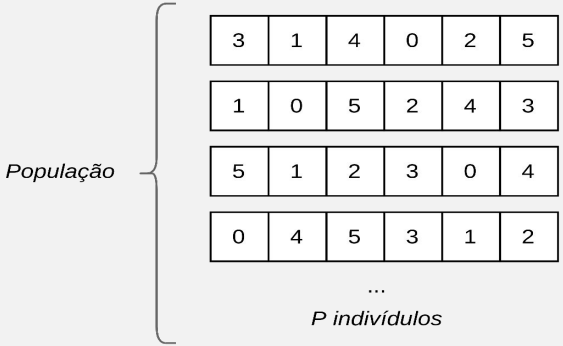


Algoritmo Genético - População Inicial

Vamos usar o Problema do Caixeiro Viajante como exemplo

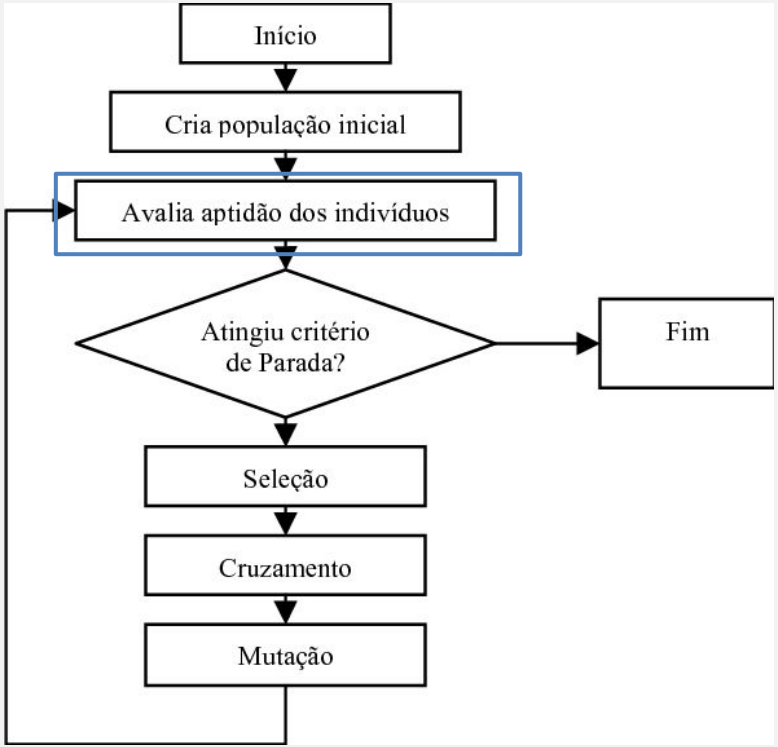
Codificação por Permutação de Inteiros: números inteiros sem repetição - sequência de visitas (garanto visita única)





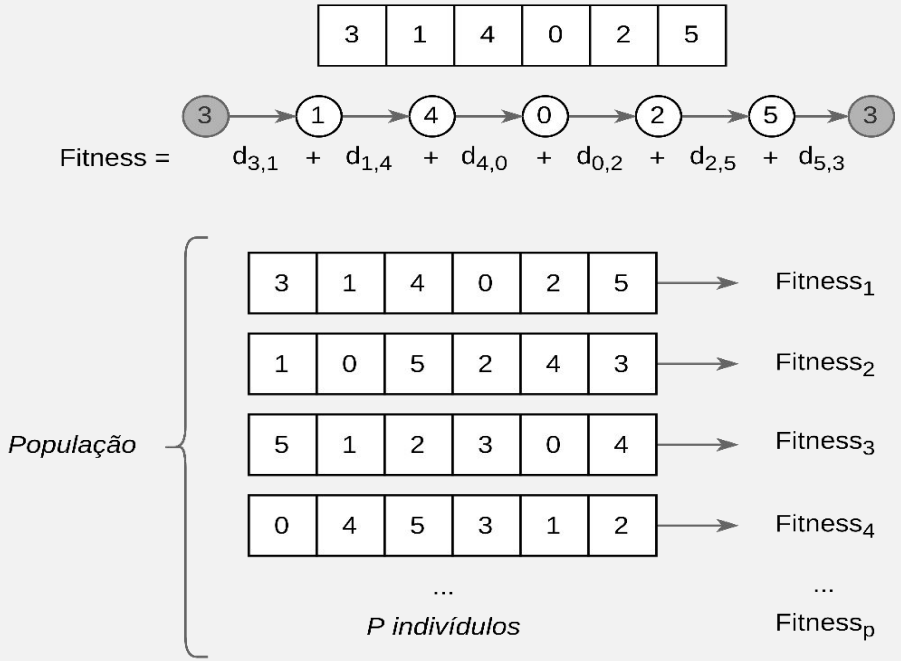
*Geração **aleatória (recomendado)** das permutações ou por meio de outras heurísticas*

Algoritmo Genético

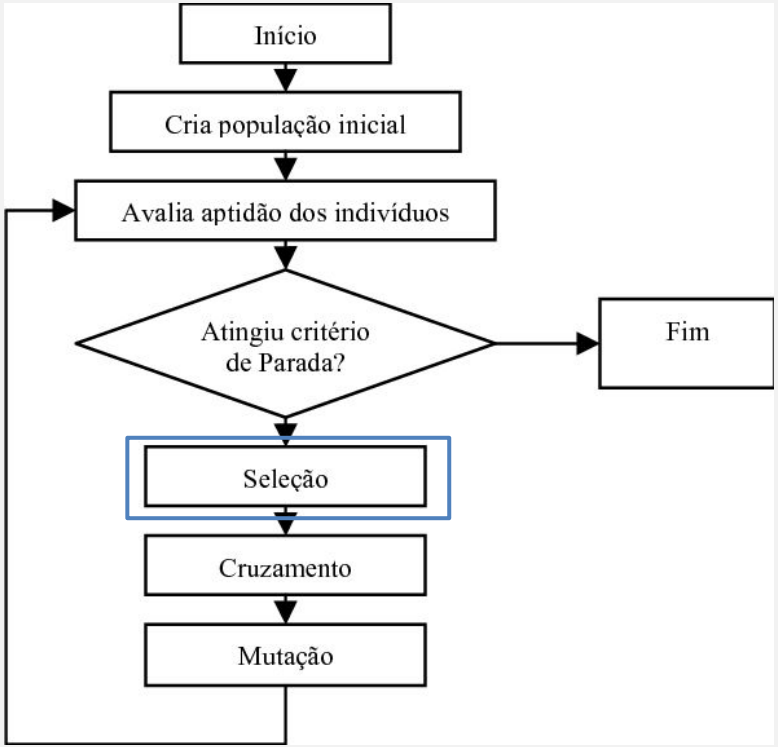


Algoritmo Genético - Fitness

Para o problema do Caixeiro Viajante o Fitness pode ser definido pela **soma das distâncias de acordo com a sequência de visitas**



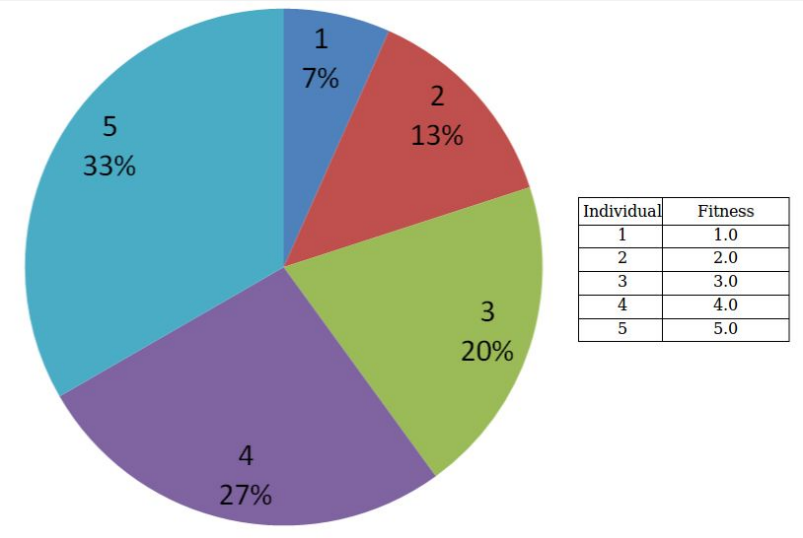
Algoritmo Genético



Algoritmo Genético - Seleção por Roleta

Preciso selecionar indivíduos pais para gerar filhos para as próximas gerações (iterações)

Seleção por Roleta: Quanto melhor o *Fitness* em relação ao objetivo do problema, maior a chance de ser escolhido para reproduzir

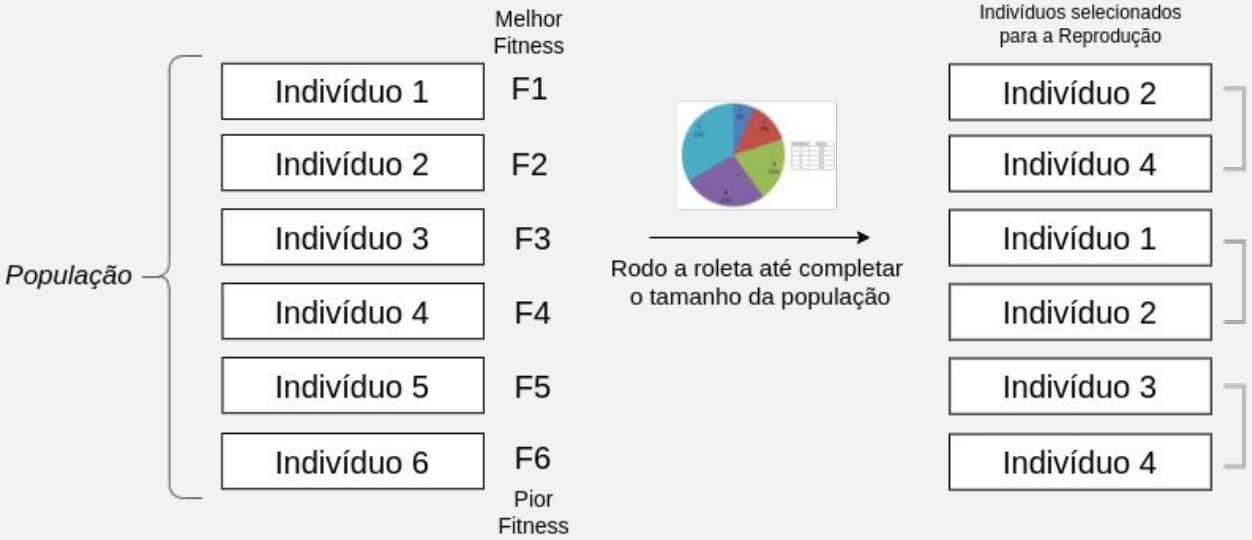


Individual	Fitness
1	1.0
2	2.0
3	3.0
4	4.0
5	5.0

Algoritmo Genético - Seleção por Roleta

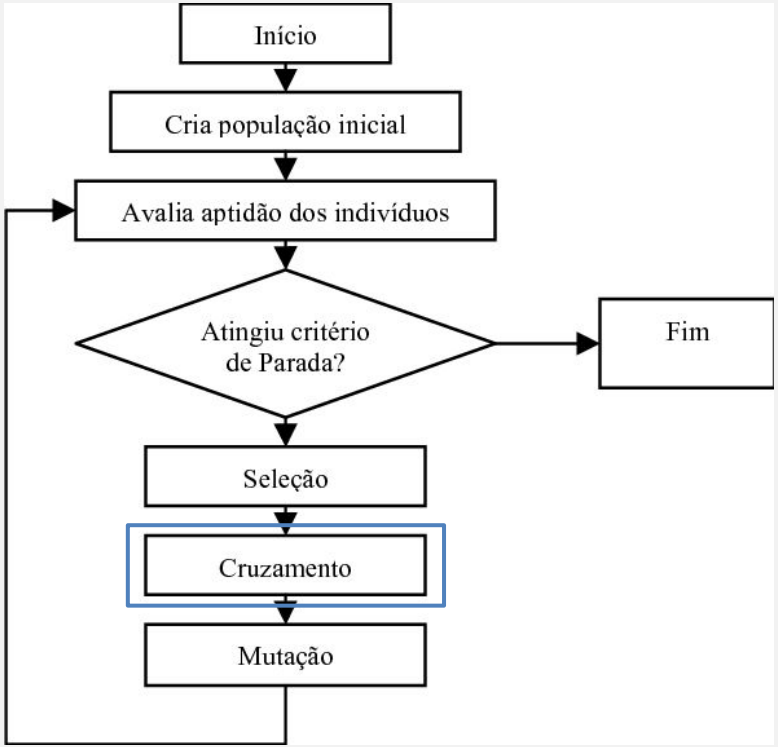
Seleção em pares até completar o tamanho da população

Resultado: P pais selecionados para a reprodução (Crossover e Mutação)



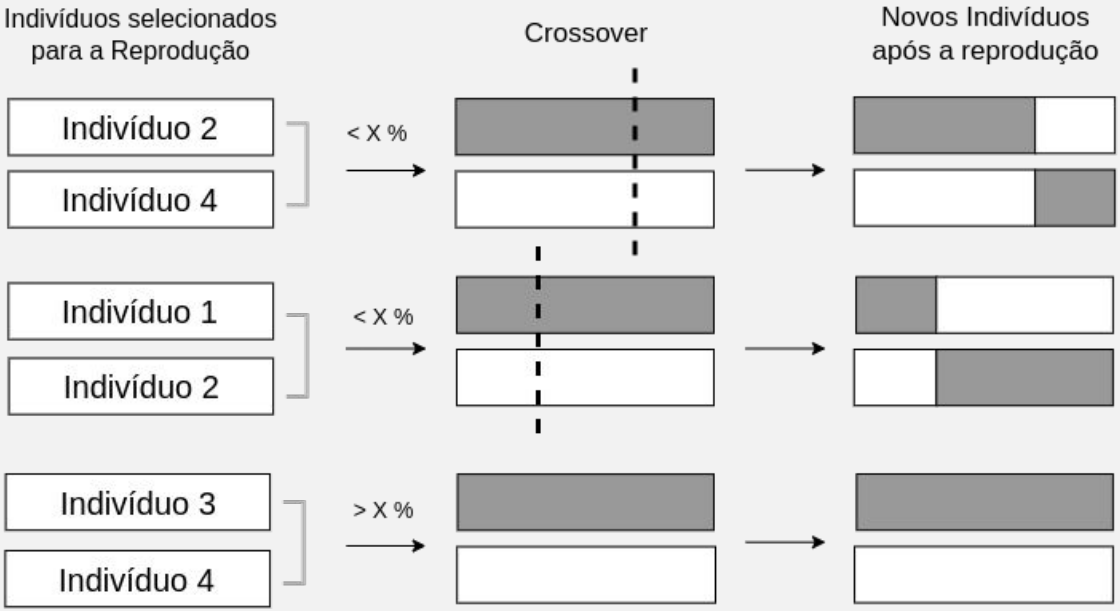
Caso o tamanho da população for ímpar, completar os pares e remover após a reprodução

Algoritmo Genético



Algoritmo Genético - Cruzamento / Crossover

Pontos de corte aleatórios (um ou mais) e combinação entre os pais para gerar filhos

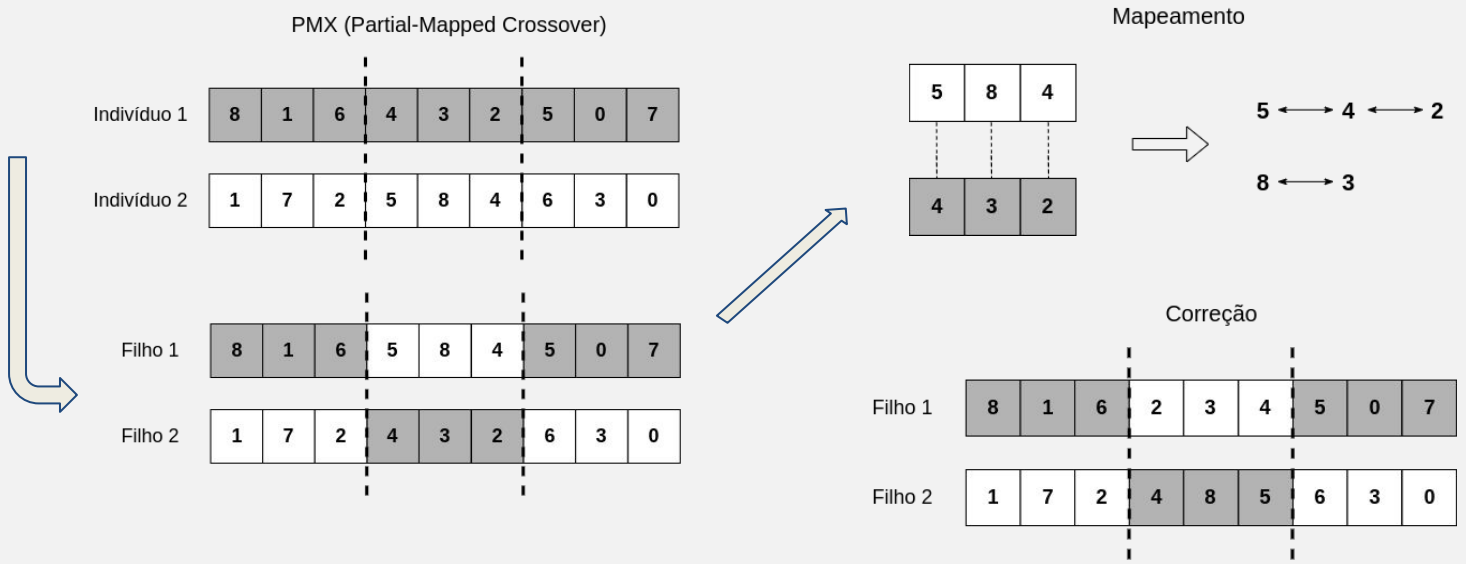


Para cada par de pais, o crossover só acontece com uma chance de X % (taxa de crossover)

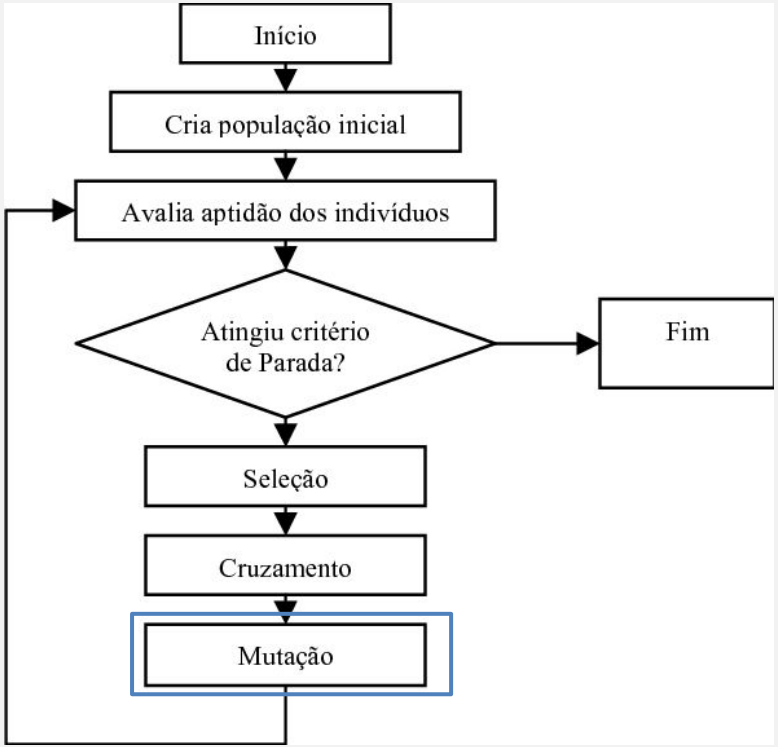
Algoritmo Genético - Cruzamento / Crossover

Para o problema do **Caixeiro Viajante** podemos usar o operador **PMX**

Operador específico para tratar de cromossomos com permutação de inteiros



Algoritmo Genético

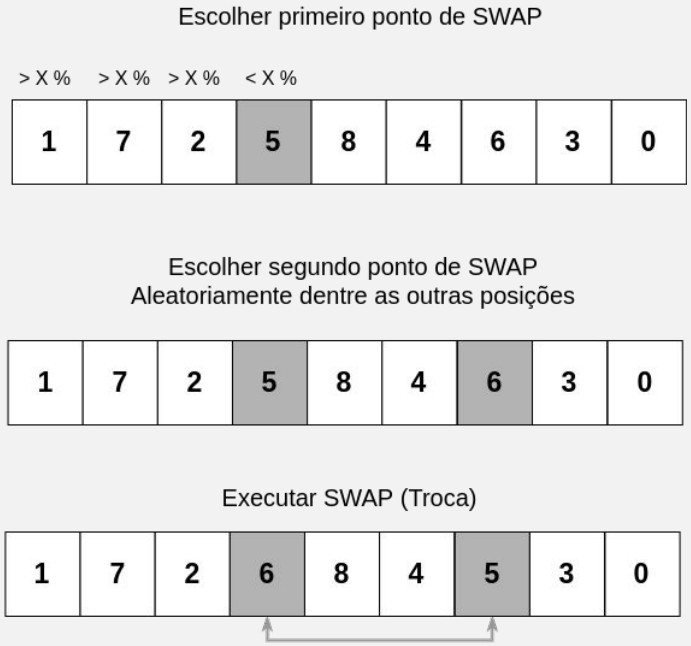


Algoritmo Genético - Mutação SWAP

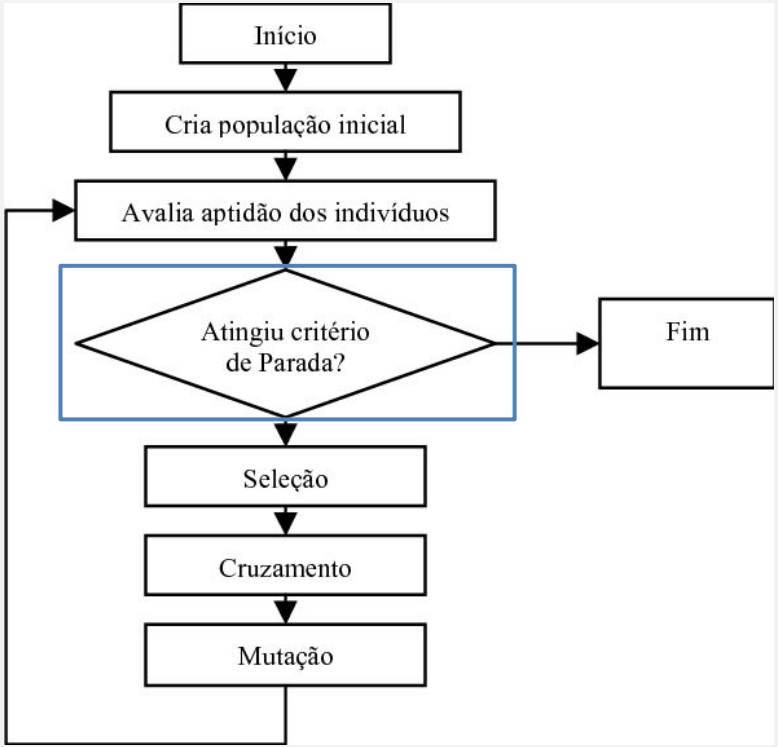
Mutação gera **variações** em um determinado indivíduo com uma chance de X % para cada Alelo

Para o Caixeiro Viajante a mutação SWAP

Executa este processo até passar por todos os Alelos de todos os indivíduos da População



Algoritmo Genético



Algoritmo Genético - Critério de Parada

- Tempo de execução (Ex: por 5 minutos)
- **Número de Gerações / Iterações (Ex: 1000 iterações)**
- Perda de diversidade
- Convergência
  - Passou k iterações sem encontrar solução melhor

Algoritmo Genético - Parâmetros

- **Tamanho da população P**
  - Populações muito pequenas (pouca diversidade) vs. população muito grande (muita diversidade, porém mais lento - muita avaliação de fitness)

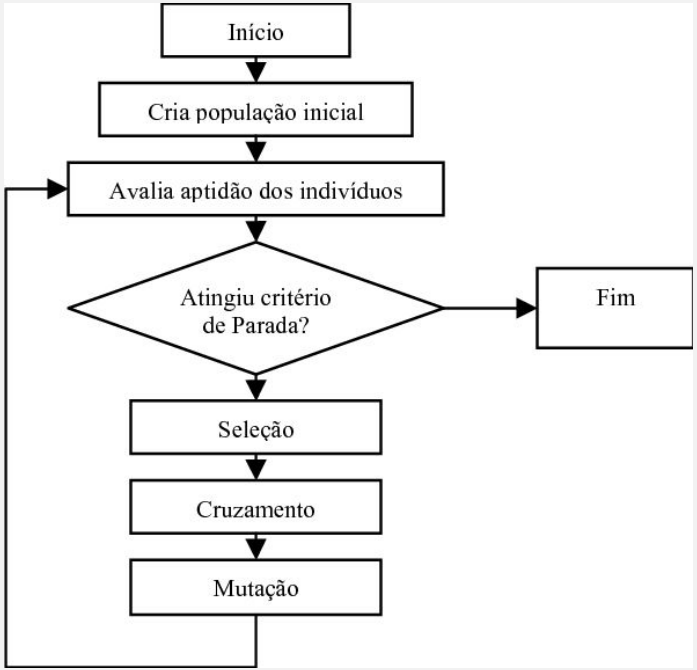
- **Critério de parada (número de iterações):**
  - Poucas iterações leva a pouca exploração do espaço de busca
  - Não tem problema ter muitas iterações, mas uma vez que convergiu torna-se desnecessário executar mais iterações

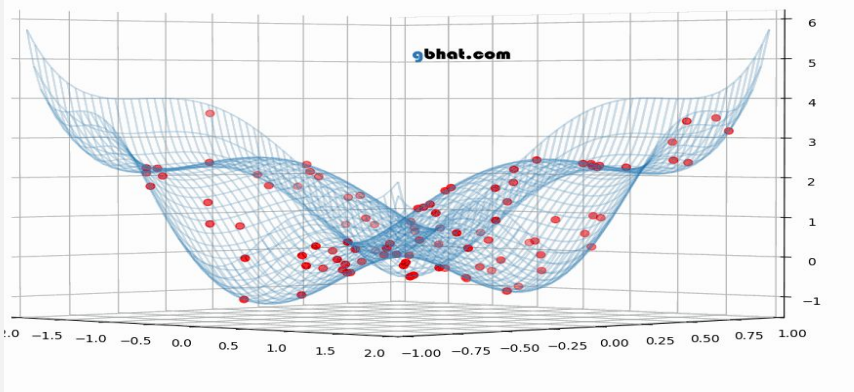
Algoritmo Genético - Parâmetros

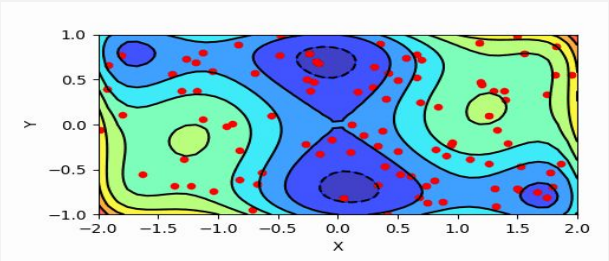
- **Taxa de crossover**
  - Quanto maior a taxa de crossover maior a chance de pais reproduzirem
  - Maior taxa = mais refinamento da solução (busca local)
  - Gera indivíduos mais especializados (pode cair em atratores locais)

- **Taxa de mutação**
  - Quanto maior a taxa de mutação maior a chance de um alelo sofrer troca
  - Maior taxa = busca aleatória, pois quase todo alelo muda e perde o propósito da reprodução (busca global)

Algoritmo Genético

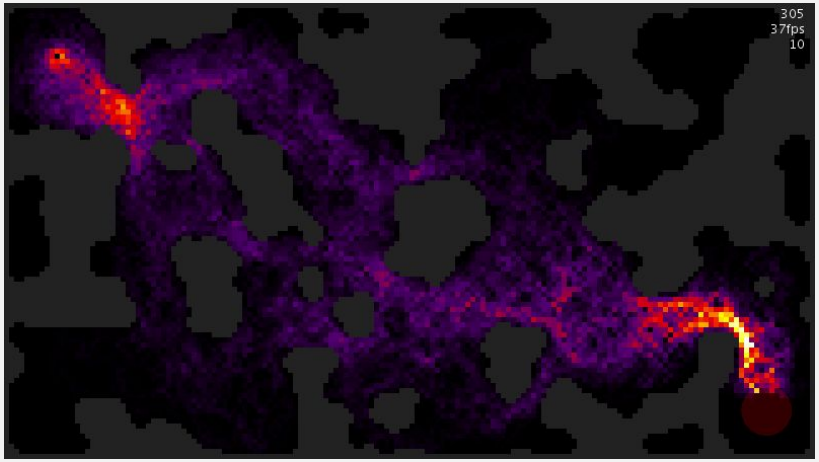




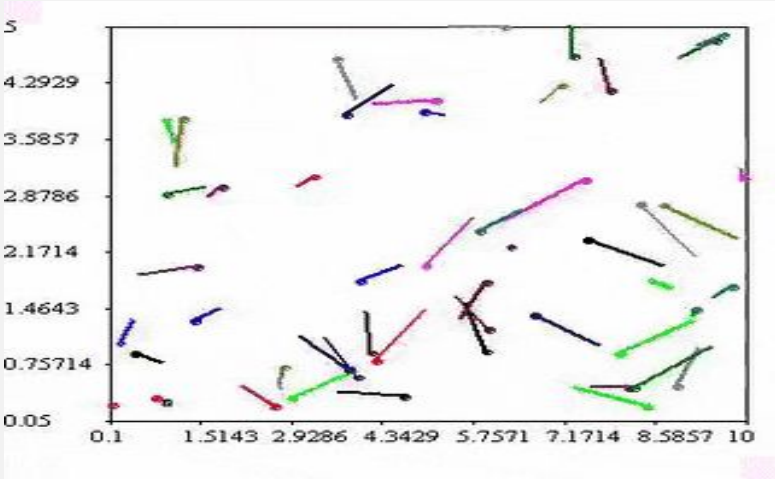


Outras Meta-heurísticas

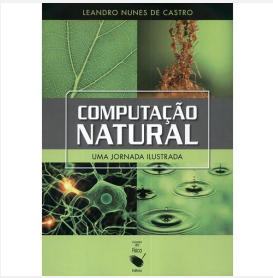
Ant Colony Optimization (ACO)



Particle Swarm Optimization (PSO)



Infinidade de outras meta-heurísticas bio-inspirados ou não



Resumo da Aula

Jornada de Modelagem e Desenvolvimento de Solução para Problema de Otimização

1. **Abstrair problema da realidade** e representação como um problema de otimização combinatória (matematicamente ou conceitualmente)

2. **Selecionar heurística ou meta-heurística** para a solução do problema

3. **Modelar a solução** baseada nas representações contidas na heurística/metaheurística

4. **Desenvolvimento** da Solução

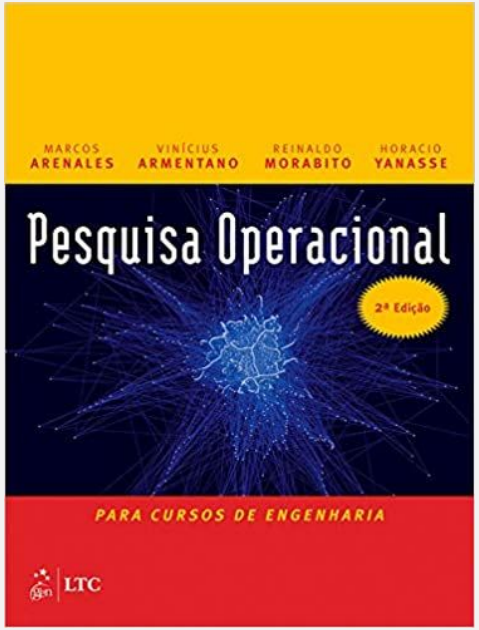
5. **Avaliação dos resultados** (Análises, Estatísticas)

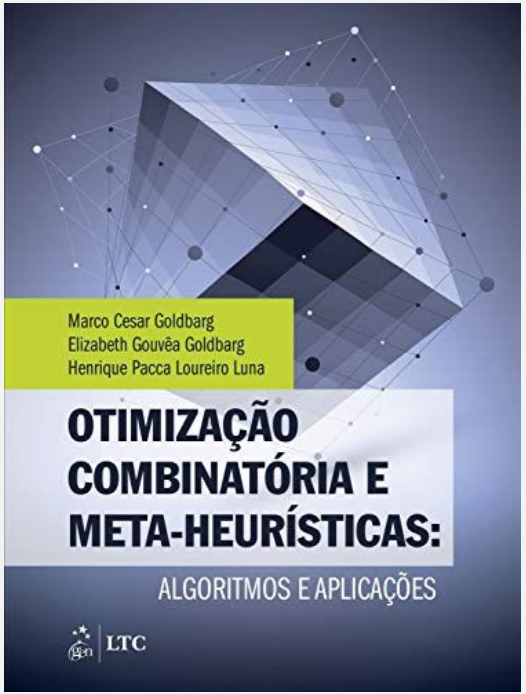
Como a área de otimização evolui?

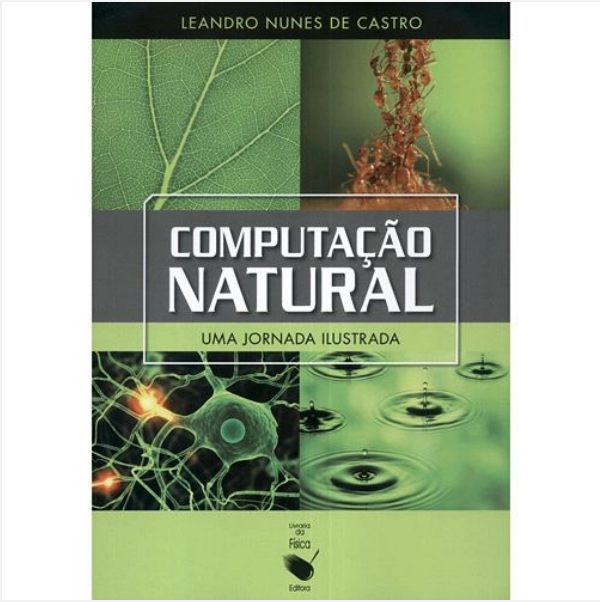
- Novas características para o problema ou junção de restrições
- Novas estratégias de busca (Heurísticas e Meta-heurísticas)
- Usar estratégias não usadas para uma problema
- Alterações em estratégias existentes
- Combinação de estratégias

Futuro: Meta-heurística genérica ? IA genérica?

Referências - Livros







Problemas de Otimização

Vítor Gauer Pereira

Programa de Pós Graduação em Computação Aplicada - PPGCA

UDESC
UNIVERSIDADE DO ESTADO DE SANTA CATARINA

Laboratório

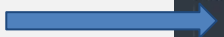
Resolver o problema do **Caixeiro Viajante** usando a heurística construtiva do **Vizinho Mais Próximo** seguido de uma heurística de **Refinamento Exchange/Swap**

1) Ler arquivo de entrada contendo as cidades
2) Calcular matriz de distância Euclidiana
3) Implementar a heurística construtiva do Vizinho Mais Próximo
4) Implementar a heurística de refinamento Exchange/Swap usando como base a solução gerada no passo 3)
5) Fazer testes com os arquivos de entrada disponíveis

Linguagem de programação: Livre!

Arquivo de Entrada

Ler a partir dessa linha (id, x, y)



```
NAME : a280
COMMENT : drilling problem (Ludwig)
TYPE : TSP
DIMENSION: 280
EDGE_WEIGHT_TYPE : EUC_2D
NODE_COORD_SECTION
  1 288 149
  2 288 129
  3 270 133
  4 256 141
  5 256 157
  6 246 157
  7 236 169
  8 228 169
  9 228 161
 10 220 169
 11 212 169
 12 204 169
 13 196 169
 14 188 169
 15 196 161
EOF
```

UDESC

Distância Euclidiana

$$d(A,\ B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

Matriz de Distância: Calcular distância de todos para todos (Grafo completo)

Construtivo - Vizinho Mais Próximo

Algorithm 14 Heurística do Vizinho mais Próximo

Require: Conjunto de cidades do problema C e a matriz de distância M

1: $S \leftarrow \{1\}$ ▷ Solução: sequência de visitas iniciando na cidade 1
2: $N \leftarrow$ todas as cidades não visitadas
3: $i \leftarrow 1$ ▷ Última cidade inserida
4: **while** $N \neq \varnothing$ **do**
5: Encontrar cidade não visitada $j \in N$ mais próxima de i
6: Inserir cidade j após a cidade i na solução S
7: Remover cidade j da lista de cidades não visitadas N
8: $i \leftarrow j$
9: **end while**
10: **return** S

Refinamento - Exchange/Swap

Algorithm 15 Heurística de Refinamento $Exchange/Swap$

Require: Conjunto de cidades C, a matriz de distância M e uma solução S

```
melhorou ← true
while melhorou do
    melhorou ← false
    for all i in S do
        for all j in S do
            S' ← cópia da solução S
            Trocar posições i e j entre si na solução S'
            if dist(S') < dist(S) then
                S ← S'
                melhorou ← true
                break
            end if
        end for
        if melhorou then
            break
        end if
    end for
end while
return S
```

UDESC